

Введение в CUDA

История развития графических процессоров



Diamond Stealth 3D 2000 (S3 ViRGE)



NVIDIA GeForce GTX 980 Ti



NVIDIA rolls out 8800 GTS 512MB graphics



NVIDIA Tesla S870

S3 ViRGE

64-битный интегрированный 2D/3D акселератор с наличием ТВ - выхода, исполнение в AGP и PCI вариантах, 2-4 Mb SGRAM (83 MHz) или EDO (66 MHz), 170 MHz RAMDAC, поддержка Direct3D, BRender, RenderWare, OpenGL и собственного API S3D, возможно использование телевизионного экрана вместо монитора

Первое поколение

Специализированный процессор для 3D графики

- буфер глубины
- аппаратное распараллеливание
- текстурирование
- задание цвета
- интерполяционная закраска

Второе поколение

- Transform & Lighting (преобразование координат вершин в плоские координаты, отображаемые на мониторе, и вычисление их освещенности)



NVIDIA GeForce 256

Третье поколение

NVIDIA GeForce 2, 3, 4; Radeon 8500 – 9200

Возможность программирования (шейдеры)

- ассемблер для граф. процессора
- ограничены по длине
- Формат с фиксированной запятой

Четвертое поколение

NVIDIA GeForce 5, 6, 7; ATI Radeon 9500 – X800

- 32-х разрядная точность
- Direct3D, OpenGL, HLSL, GLSL, Cg
- начало GPGPU (General Purpose GPU)

Пятое поколение

NVIDIA GeForce 8 – GTX 200;

ATI Radeon X 1K – HD 5K

- расширенные возможности программирования (унифицированные шейдеры, CUDA, AMD FireStream)

GPGPU

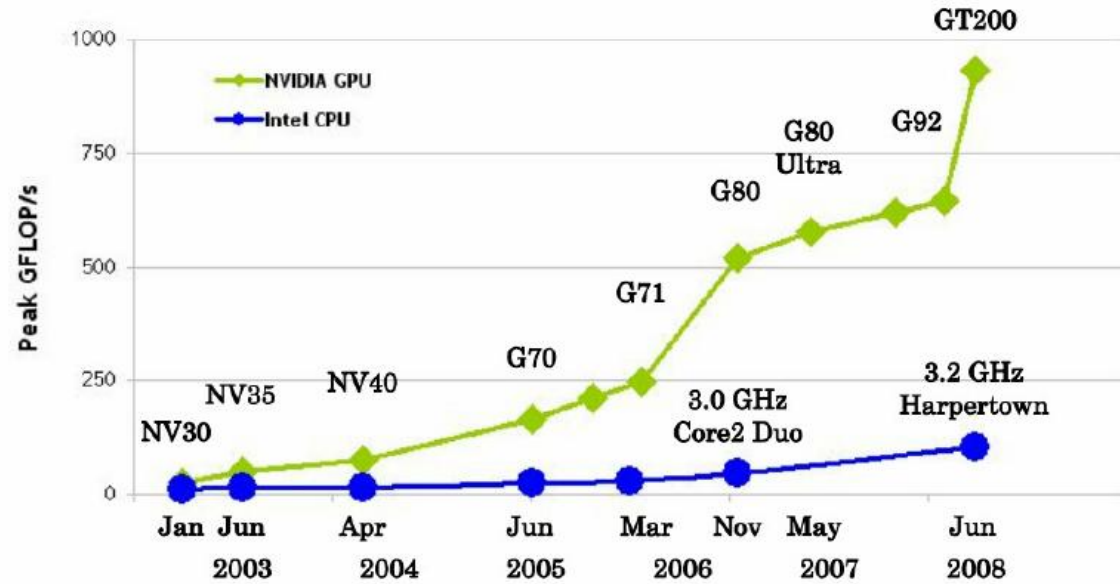
General Purpose computing on Graphical Processing Units

- assembler
- Шейдеры
- OpenGL, DirectX
- CUDA, ATI FireStream, OpenCL

Частоты CPU

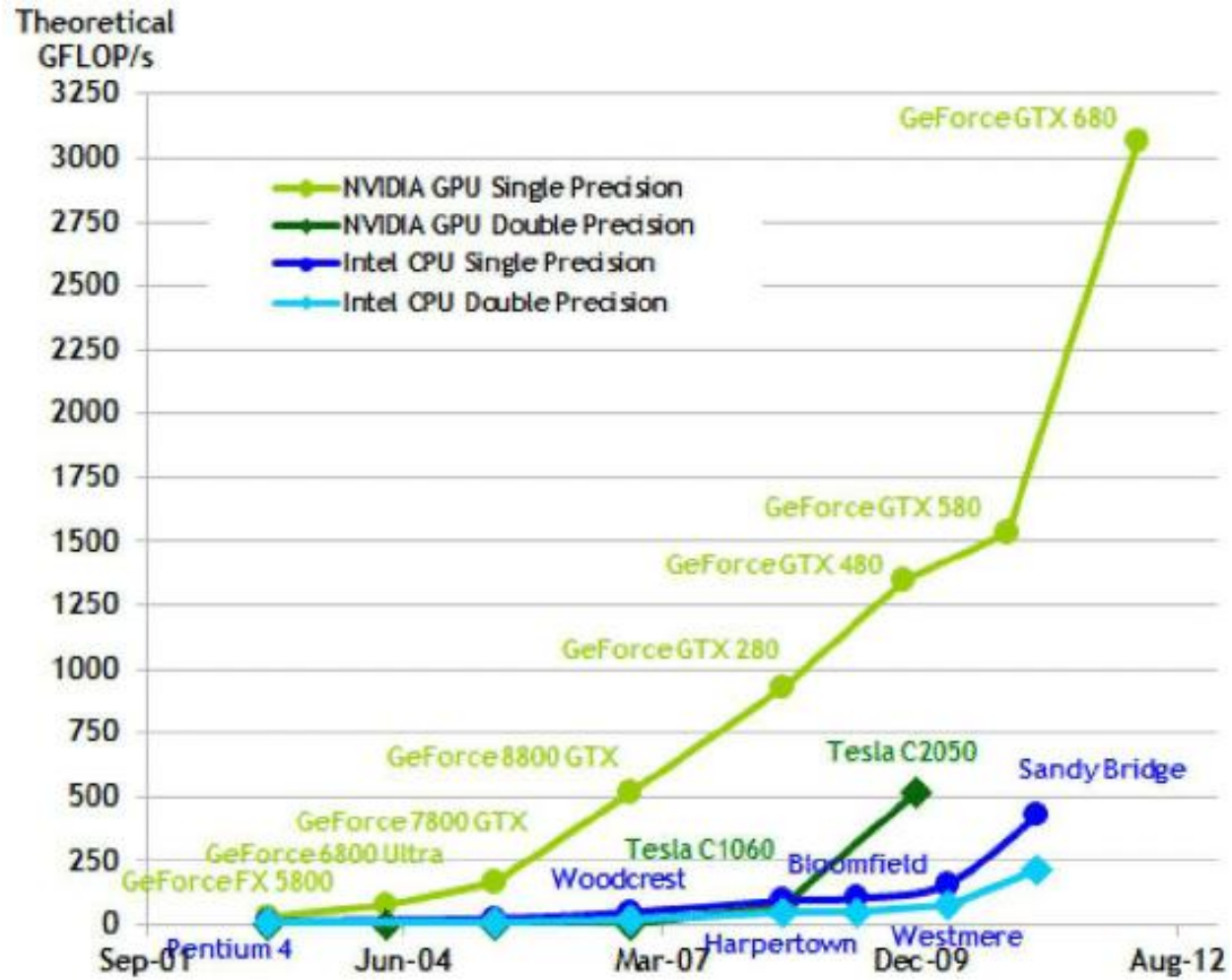
- 2004 г. - Pentium 4, 3.46 GHz
- 2005 г. - Pentium 4, 3.8 GHz
- 2006 г. - Core Duo T2700, 2333 MHz
- 2007 г. - Core 2 Duo E6700, 2.66 GHz
- 2007 г. - Core 2 Duo E6800, 3 GHz
- 2008 г. - Core 2 Duo E8600, 3.33 Ghz
- 2009 г. - Core i7 950, 3.06 GHz
- 2013 г. – Core i7 3970x, 3.5 GHz

Floating point operations per second

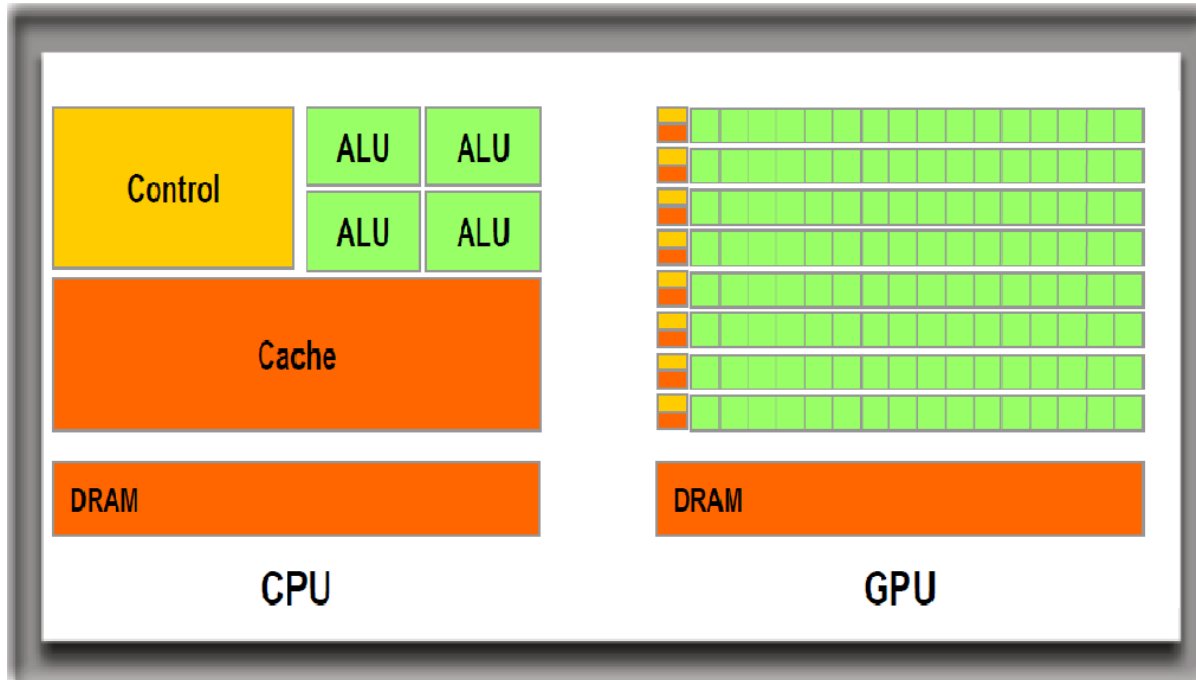


GT200 = GeForce GTX 280	G71 = GeForce 7900 GTX	NV35 = GeForce FX 5950 Ultra
G92 = GeForce 9800 GTX	G70 = GeForce 7800 GTX	NV30 = GeForce FX 5800
G80 = GeForce 8800 GTX	NV40 = GeForce 6800 Ultra	

Floating point operations per second



Разница между CPU и GPU



CUDA = Compute Unified Device Architecture

- Многократное ускорение в каждой конкретной задаче достигается в результате усилий программиста
- Существовали ранние попытки использования графических карт для научных расчётов
- Проблема – алгоритмы должны были быть реализованы на специальном “шейдерном” языке (shade language)
- Для облегчения работы с алгоритмами общего назначения
- компания NVIDIA выдвинула инициативу создания аппаратно/программной архитектуры общего назначения

Общие положения CUDA

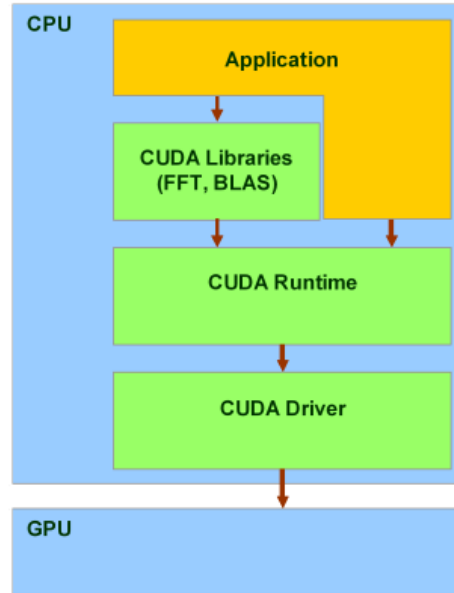
- GPU – **сопроцессор** для CPU (**хоста**)
- У GPU есть собственная память (**device memory**)
- GPU способен одновременно обрабатывать множество процессов (**threads**) данных одним и тем же алгоритмом
- Для осуществления расчётов при помощи GPU хост должен осуществить запуск вычислительного ядра (**kernel**), который определяет конфигурацию GPU в вычислениях и способ получения результатов (алгоритм)
- Процессы GPU (в отличие от CPU) очень просты и многочисленны

Возможности NVIDIA CUDA

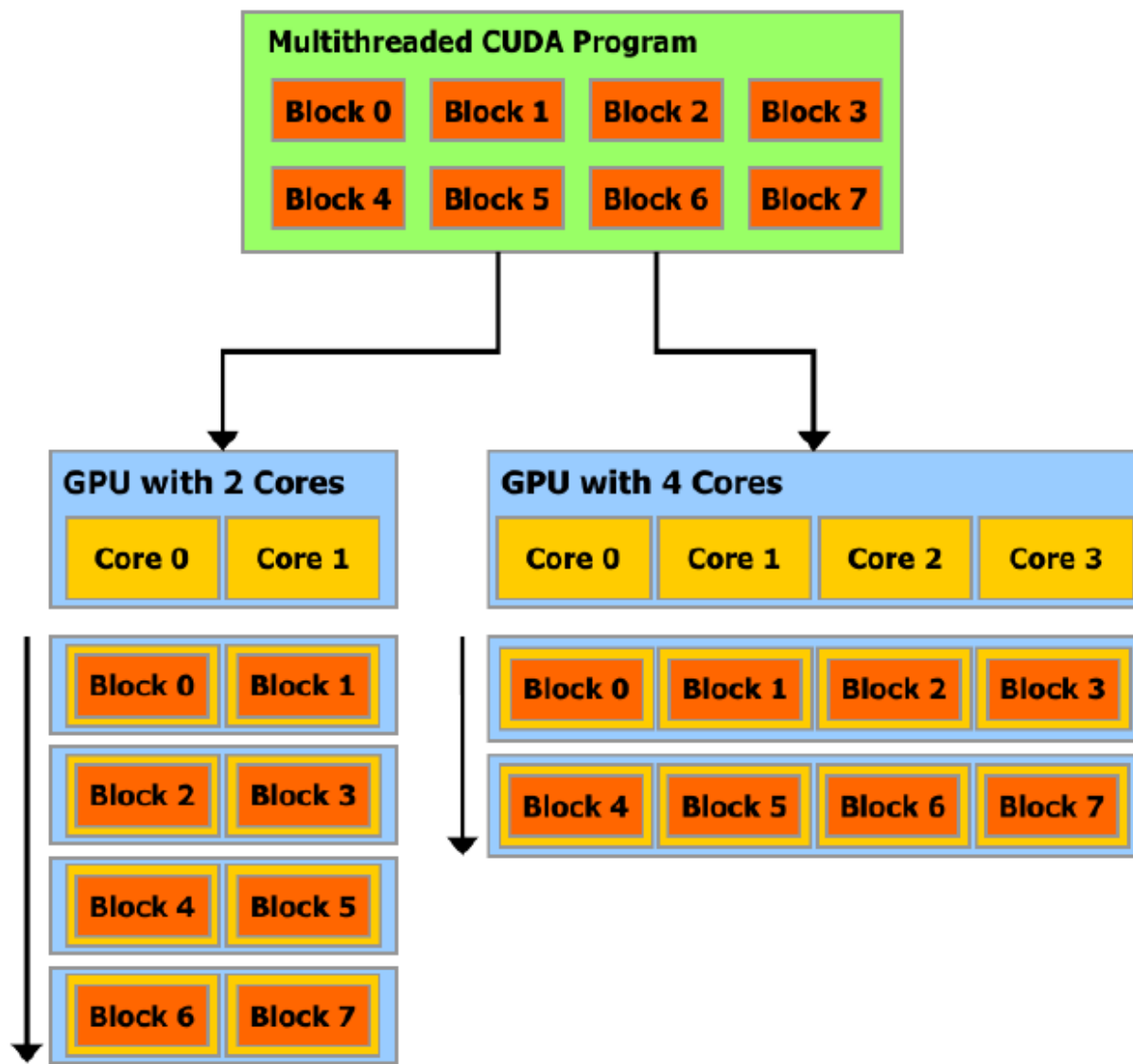
- унифицированное программно-аппаратное решение для параллельных вычислений на видеочипах NVIDIA
- большой набор поддерживаемых решений, от мобильных до мультичиповых
- стандартный язык программирования Си
- дополнительные библиотеки работающие с GPU
- оптимизированный обмен данными между CPU и GPU
- взаимодействие с графическими API OpenGL и DirectX
- поддержка 32- и 64-битных операционных систем: Windows XP, Windows Vista, Linux и MacOS X
- возможность разработки на низком уровне

Состав CUDA

- CUDA включает два API: высокого уровня (CUDA Runtime API) и низкого (CUDA Driver API)
- **CUBLAS** — CUDA вариант BLAS (Basic Linear Algebra Subprograms), предназначенный для вычислений задач линейной алгебры и использующий прямой доступ к ресурсам GPU;
- **CUFFT** — CUDA вариант библиотеки Fast Fourier Transform для расчёта быстрого преобразования Фурье, широко используемого при обработке сигналов.



Распараллеливание CUDA



SIMD подход

Single Instruction Multiple Data - группа параллельно работающих процессоров осуществляют действия над разными данными, но при этом все они в произвольный момент времени должны выполнять одинаковую команду

Классификация по Флинну

	Single Instruction	Multiple Instruction
Single Data	SISD	MISD
Multiple Data	SIMD	MIMD

Классы систем

- CPU (одноядерный) — SISD (одновременно выполняется только одна инструкция над одним набором операндов);
- CPU (многоядерный) — MIMD (Одновременно несколько ядер могут работать совершенно независимо, каждое как SISD);
- GPU (NVIDIA ComputeCapability версии < 2.0) — SIMD (одновременно на графическом адаптере может выполняться только один поток вычислений, который работает с большим набором данных);
- GPU (NVIDIA ComputeCapability версии ≥ 2.0) — MIMD (одновременно на графическом адаптере может выполняться несколько потоков вычислений, каждый из которых работает с большим набором данных).

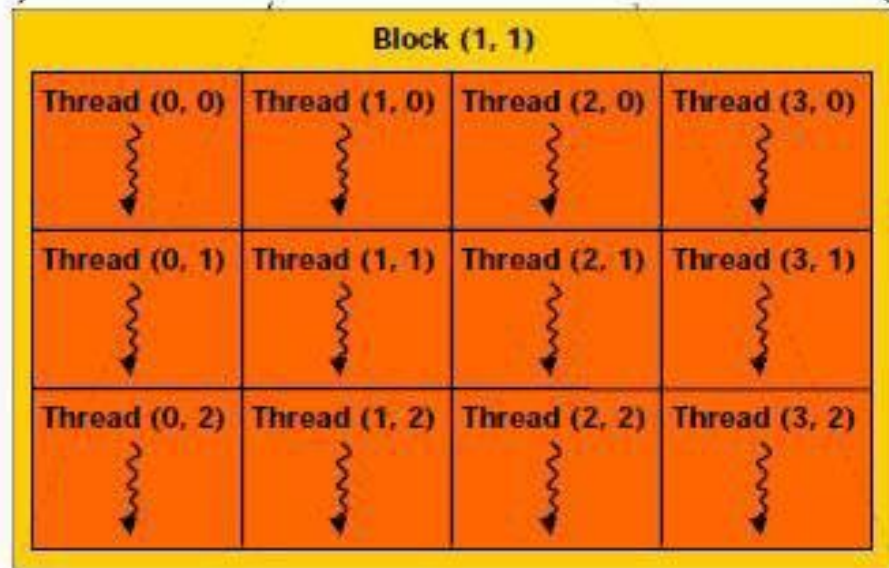
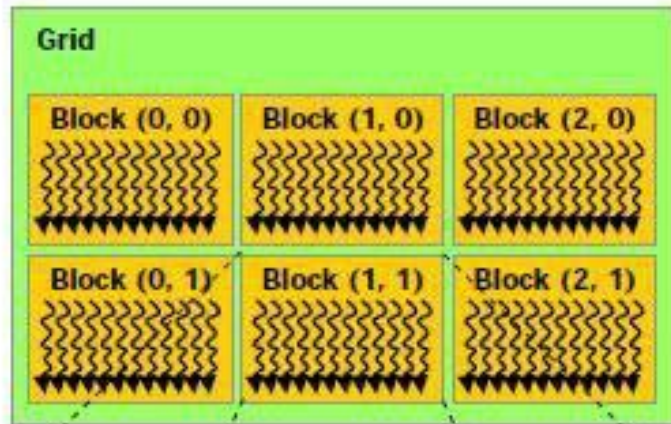
Графические процессоры изначально предназначены для параллельного решения одной массивно-параллельной задачи

Ограничения CUDA

- Отсутствие поддержки рекурсии для выполняемых функций на первых видеокартах
- Закрытая архитектура
- Поддержка только устройств NVIDIA

Программная модель

- Хост (Host) — центральный процессор, управляющий выполнением программы.
- Устройство (Device) — видеоадаптер, выступающий в роли сопроцессора центрального процессора.
- Тред (Thread, поток) — единица выполнения программы. Имеет свой уникальный идентификатор внутри блока.
- Блок (Block) — объединение тредов, которое выполняется целиком на одном SM. Имеет свой уникальный идентификатор внутри грида.
- Грид (Grid) — объединение блоков, которые выполняются на одном устройстве.
- Ядро (Kernel) — параллельная часть алгоритма, выполняется на гриде.
- Варп (Warp) — 32 последовательно идущих треда, выполняется физически одновременно.



**C Program
Sequential
Execution**

Serial code

Parallel kernel
Kernel0<<<>>>()

Serial code

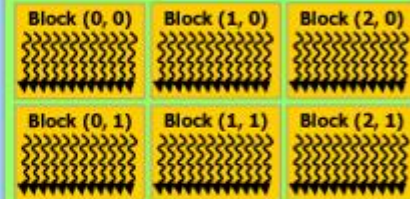
Parallel kernel
Kernel1<<<>>>()

Host



Device

Grid 0

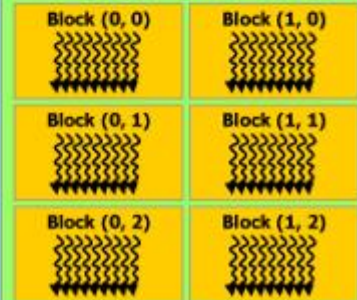


Host



Device

Grid 1



Введение в CUDA